introduction

Each checker module is verified independently before the full-pipeline testbench is run. The testbench for a given ch

The full-pipeline testbench (`ids_top_tb.vhd`) then exercises the entire chain from RMII dibit injection to alert latch

A critical testbench optimisation is the use of a reduced clock frequency generic:

[caption=Generic override in the testbench, label=lst:generic] $u_dut : entity work.ids_top generic map(CLK_FREQ_HZ$

At `CLK_FREQ_HZ = 1000`, the frame counter's one-second tick fires after 1000 clock cycles instead of 50 000 000. A 50

Test Procedures

The testbench uses VHDL procedures to encapsulate frame injection. Each procedure drives the RMII signals cycle

[caption=Injecting a land attack in the testbench, label=lst:landattack] $send_ipv4_frame(src_ip => x"C0A80101", -$

IPv4 Checker

All twelve R1–R12 rules were verified in a single simulation run by injecting a sequence of frames, each crafted to t

*Header Validation Rules (R1–R7) The following rules were confirmed to fire at the correct simulation time:

R1: Frame with `ip_version = 6` — `alert_bad_version` asserted.
R2: Frame with `ip_ihl = 3` — `alert_bad_ihl` asserted.
R3: Frame with `ip_length = 10` — `alert_bad_length` asserted.
R4: Frame with `ip_ttl = 1` — `alert_low_ttl` asserted.
R5: Frame with non-zero fragment offset — `alert_fragment` asserted.
R6: Frame with `ip_src = 0.0.0.0` — `alert_spoof_src` asserted.
R7: Frame with `ip_src = ip_dst` — `alert_land_attack` asserted.

*Bogon Address Rules (R8–R10) Bogon checks operate on the upper bits of `ip_src` and fire as soon as the source a

R8: Frame with `ip_src = 127.0.0.1` (`ip_src[31:24] = 0x7F`) — `alert_bogon_loop` asserted.
R9: Frame with `ip_src = 169.254.0.1` (`ip_src[31:16] = 0xA9FE`) — `alert_bogon_link` asserted.
R10: Frame with `ip_src = 224.0.0.1` (`ip_src[31:28] = 0xE`) — `alert_bogon_mcast` asserted.

*Header Abuse Rules (R11–R12)

R11: Frame with `ip_ihl = 6` (options field present, header exceeds minimum 20 bytes) — `alert_ip_options` asserted.
R12: Frame with the reserved IPv4 flag bit set (`ip_flags[2] = '1'`) — `alert_reserved_bit` asserted.

In all twelve cases, no secondary alert was triggered on frames designed to exercise a single rule, confirming that th

TCP Checker Verification

Rules T1–T5 were verified by injecting five TCP frames with specific flag values and destination ports. Key test cas

T1: `tcp_flags = 0x03` (SYN+FIN) — `alert_syn_fin` asserted.
T2: `tcp_flags = 0x06` (SYN+RST) — `alert_syn_rst` asserted.
T3: `tcp_flags = 0x00` (NULL) — `alert_null_scan` asserted.
T4: `tcp_flags = 0xFF` (all flags) — `alert_xmas_scan` asserted.
T5: `dst_port = 23` (Telnet) — `alert_forbidden` asserted.

[H] [width=]$tcp_checker_waveform.png Simulation waveforms showing TCP rules T1 - -T5 firing in sequence.$ UDP C

All nine U1–U9 rules were verified by injecting dedicated UDP frames, each violating exactly one rule while keeping

*Forbidden Port and Structural Rules (U1–U3)

U1: Frame with `udp_dst_port = 69` (TFTP, in the configured forbidden port table) — `alert_udp_forbidden` asserted.
U2: Frame with `udp_length = 4` (below the minimum valid value of 8) — `alert_udp_short` asserted.
U3: Frame with `udp_length = 0` — `alert_udp_zero` asserted.

*Amplification and Spoofing Rules (U4–U5)

U4: Frame with `udp_src_port = 53` (DNS response arriving from an amplification reflector) — `alert_udp_amplify` ass
U5: Frame with `ip_src = ip_dst` and `udp_src_port = udp_dst_port` (UDP land attack, both IP-layer and port-layer c

*Size and Service Abuse Rules (U6–U9)

U6: Frame with `udp_length = 1480` (exceeding the 1472-byte MTU threshold) — `alert_udp_oversize` asserted.
U7: Frame with `udp_dst_port = 0` (reserved port zero, never valid in legitimate traffic) — `alert_udp_port_zero` assert
U8: Frame with `udp_dst_port = 1900` (SSDP/UPnP, a common amplification vector) — `alert_ssdp_abuse` asserted.
U9: Frame with `udp_dst_port = 11211` (Memcached amplification request) — `alert_memcached` asserted.

In all nine cases, the alert signal was asserted within the expected number of clock cycles from the relevant header